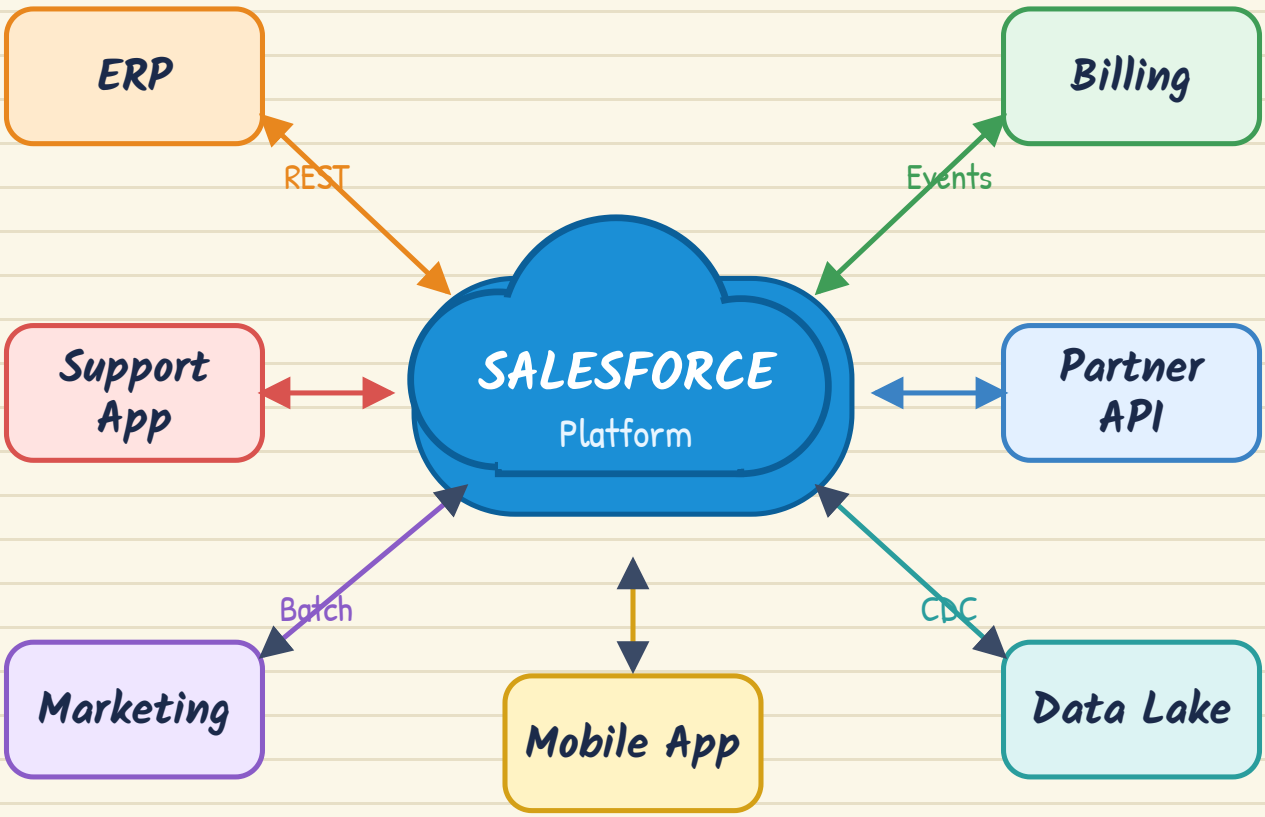
# SALESFORCE

## COMPLETE NOTES

## (Integration)




**BEGINNER TO ADVANCED**

SWIPE TO NEXT →

# Roadmap

APIs
Integration

*Everything you need, one concept per page*


3 What is Salesforce Integration?

4 The API Landscape

5 Authentication: OAuth & ECAs

6 REST API: CRUD & SOQL

7 Composite & Bulk API 2.0

8 Integration Patterns

9 Callouts & Named Credentials

10 Events: Platform Events, CDC, Pub/Sub

11 Apex REST: Inbound Endpoints

12 Salesforce Connect & Middleware

13 Limits & Governors

14 Errors, Retries, Idempotency

15 Security Best Practices

16 Testing & Tooling

17 Real-World Projects

18 Cheat Sheet & Interview Guide


## How to use these notes

- Read one page, then ***do the example*** in a Developer Edition org or sandbox.
- Say the ***Remember*** line out loud from memory.
- Draw the diagram on paper without looking.
- Finish with the cheat sheet and interview questions on page 18.

## Mindset

Pick the ***pattern*** first, then the API.
Estimate volume and limits ***before*** you build.
Treat every call as something that can fail.

## Tools you will need

- A Salesforce ***Developer Edition*** org or sandbox
- Postman (or curl) for API calls
- Salesforce CLI (`sf`) and VS Code with the Salesforce extensions
- An editor for Apex and JSON

## Fast-changing topics

Authentication rules and API versions change every release. Pages 4, 5 and 15 were checked against public sources in ***Oct 2026***. Re-check the Salesforce release notes before relying on dates.

**Real-world use cases**
Planning a Salesforce project that talks to ERP, billing or analytics tools.

**Interview tip**
Learn the language: callout, event, idempotency, governor limit.

**Pro tip**
Keep a notes file of every API version and limit you rely on.

# What is Salesforce Integration?

APIs
Integration

*Connecting Salesforce with other systems so data and processes flow both ways*

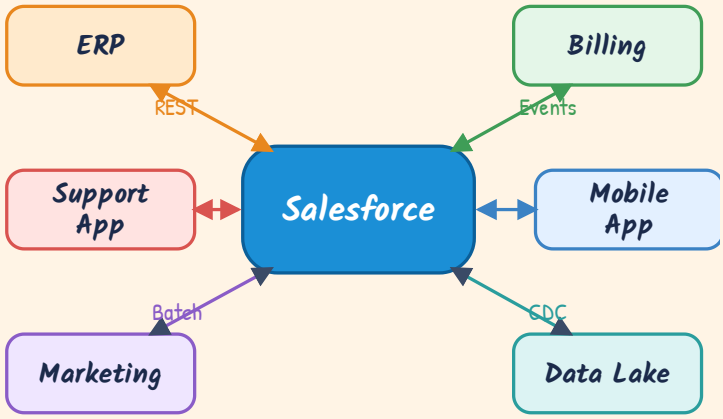
## 1 What is it?

Salesforce rarely works alone. ***Integration*** is any design where Salesforce exchanges ***data*** or triggers ***actions*** with other systems: ERP, billing, marketing, data warehouse and apps.

## 2 Why integrate?

- Stop copy-pasting between systems
- One customer view across tools
- Automate cross-system processes (order to cash)
- Feed analytics and AI with fresh data

## 3 The big picture

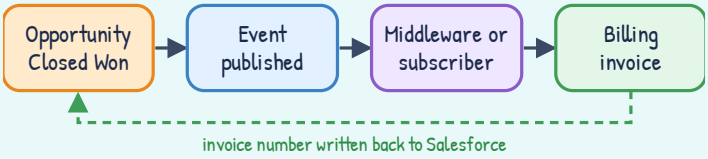



## 4 Directions and styles

| Direction | Style | Example |
|---|---|---|
| Inbound | Request / response | ERP updates an Account via REST |
| Outbound | Callout | Apex asks a tax service for a rate |
| Either way | Event-driven | Order event, billing reacts |
| Either way | Batch | Nightly load of 2M records |

## 5 Worked example

**"When an Opportunity is Closed Won, create an invoice in billing."**




## 6 Ask before you build

- ***Direction:*** in, out or both?
- ***Volume:*** 100 records or 10 million?
- ***Latency:*** seconds, minutes or nightly?
- ***Owner:*** which system is the source of truth?
- ***Failure:*** what happens when a call fails?

## 7 Common mistakes

- Point-to-point links for everything. Fix: use middleware or events when 3+ systems are involved.
- Polling all day. Fix: subscribe to events.
- Ignoring limits at design time. Fix: estimate volume first.

## 8 Remember

**Pick the pattern first, then the API.**

**✓ Real-world use cases**
ERP sync, billing, marketing automation, data warehouse feeds.

**★ Interview tip**
Be ready to name 3 integration styles and when each fits.

**➤ Pro tip**
Write down direction, volume and latency before choosing a tool.

# The API Landscape

APIs
Integration

*Which Salesforce API for which job*

## 1 What is a Salesforce API?

Salesforce exposes data and configuration through several ***API families***. Each is tuned for a different job. Every API call is made against a numbered ***version***: Summer '26 is ***v67.0***; Winter '27 is ***v68.0***.

## 2 The API families

| API | Best for | Note |
|---|---|---|
| REST | Apps, CRUD, SOQL, JSON | Default choice |
| SOAP | WSDL-based enterprise tools | `login()` retires Summer '27; use OAuth |
| Bulk API 2.0 | Large async loads and exports | CSV jobs |
| Composite / Collections | Many operations, one call | Fewer round trips |
| GraphQL | Related records in one query | You shape the response |
| Pub/Sub API | Publish and subscribe to events | gRPC |
| Metadata / Tooling | Config, deploy, dev tools | Not for business data |

## 3 Which API do I use?

| I need to... | Use |
|---|---|
| Read or write a few records | ***REST*** |
| Load 100k+ records | ***Bulk API 2.0*** |
| Do many small steps in one trip | ***Composite*** |
| Be told when data changes | ***Pub/Sub API + CDC*** |
| Deploy configuration | ***Metadata API*** |
| Support a legacy WSDL tool | ***SOAP*** (with OAuth) |

## 4 Anatomy of a REST URL

`https://MyDomain.my.salesforce.com/services/data/v67.0/sobjects/Account/001xx0000012345`

| Your org (My Domain) | REST root | API version | Resource | Record Id |
|---|---|---|---|---|

## 5 Sync, async or push?

- ***Sync:*** answer comes back now (REST, SOAP, Composite)
- ***Async:*** start a job, check later (Bulk API 2.0)
- ***Push:*** Salesforce notifies you (events)

Choose by ***volume*** and ***latency***, not habit.

## 6 Version tip

Pin the API version in your code and upgrade on purpose. A new version arrives with every release, and older versions are retired on a published schedule.

## 7 Common mistakes

- Using Bulk API for 10 records. Fix: REST or Collections.
- Polling REST in a loop. Fix: use events.
- Hard-coding an old API version. Fix: one config value.

## 8 Remember

***Small and now: REST.***
***Big and later: Bulk.***
***Tell me when: Events.***

**Real-world use cases**
Picking the right API for a new integration design.

**Interview tip**
Compare REST, SOAP and Bulk by volume, format and sync vs async.

**Pro tip**
Call `/services/data/` to list the API versions your org supports.

# Authentication: OAuth & External Client Apps

APIs
Integration

*How an external system proves who it is*

## 1 What is it?

**OAuth 2.0** lets a system get a temporary **access token** instead of sharing a username and password. The system is registered in the org as an **External Client App (ECA)**.

## 2 Why it matters

- No passwords stored in integrations
- Tokens are scoped and revocable
- The required direction for all new integrations

## 3 Client Credentials request

```
POST https://<MyDomain>.my.salesforce.com
     /services/oauth2/token

grant_type=client_credentials
client_id=<consumer key>
client_secret=<consumer secret>
```

Setup: **Setup** → **External Client App Manager**. Enable the flow and choose a **run-as** user.

## 4 Which flow?

| Flow | Use when |
|---|---|
| Client Credentials | Server to server, no user |
| JWT Bearer | Server to server with a certificate |
| Web Server + PKCE | A real user signs in |
| Username-Password / SOAP `login()` | Legacy. Not for new work |

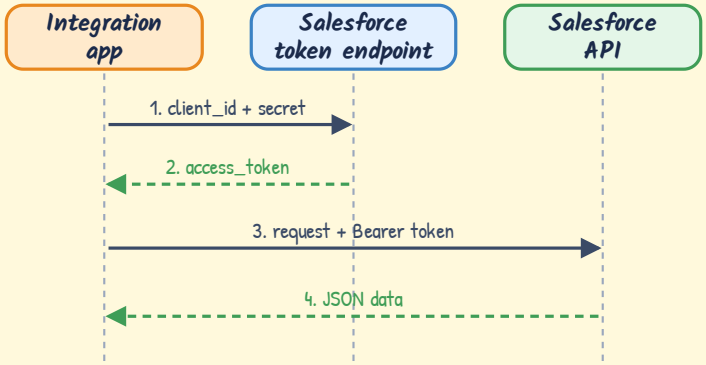
## 5 Response, then API call

**Response shape (sample)**

```
{ "access_token": "00D...!AQ...",
  "instance_url":
"https://myorg.my.salesforce.com",
  "token_type": "Bearer" }
```

Send `Authorization: Bearer <token>` with every call.

## 6 The conversation




## 7 Common mistakes

- Looking for "New Connected App" and finding it disabled. Fix: create an ECA.
- Secret in source code. Fix: secrets manager or env variable.
- Integration user is a System Administrator. Fix: minimal permission set.

## 8 What is changing (check release notes)

New Connected Apps are disabled by default since **Spring '26**; ECAs are preferred. SOAP `login()` retires in **Summer '27**. The OAuth username-password flow is being retired, but the exact date has shifted, so verify it.

## 9 Remember

**Token, not password. ECA, not Connected App.**

**Real-world use cases**
ETL tools, middleware, mobile backends, scheduled jobs.

**Interview tip**
Why does Client Credentials need a run-as user?

**Pro tip**
Request only the OAuth scopes you need and rotate secrets on a schedule.

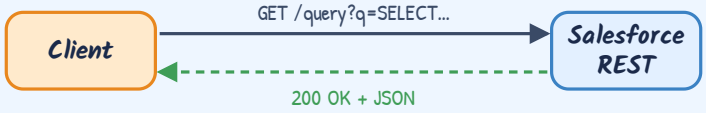
# REST API: CRUD & SOQL

APIs Integration

*Read and write records over HTTPS and JSON*

## 1 What is it?

The **REST API** reads and writes Salesforce data over HTTPS using JSON. It is the default choice for apps, scripts and middleware.




## 2 CRUD on one record

| Action | HTTP | URL |
|---|---|---|
| Create | POST | /sobjects/Account |
| Read | GET | /sobjects/Account/{Id} |
| Update | PATCH | /sobjects/Account/{Id} |
| Delete | DELETE | /sobjects/Account/{Id} |
| Upsert | PATCH | /sobjects/Account/ExtId__c/{value} |
| Query | GET | /query?q=SOQL |

## 3 Create a record

**Request body**

```
POST /services/data/v67.0/sobjects/Account
{ "Name": "Acme Corp" }
```

**Response shape (sample)**

```
{ "id": "001xx0000012345AAA",
  "success": true, "errors": [] }
```

## 4 Query with SOQL

```
GET /services/data/v67.0/query
    ?q=SELECT+Id,Name+FROM+Account+LIMIT+2
```

**Response shape (sample)**

```
{ "totalSize": 2, "done": true,
  "records": [ { "attributes": {...},
    "Id": "001...", "Name": "Acme" } ] }
```

## 5 SOQL basics

```
SELECT Name, Industry
FROM Account
WHERE Industry = 'Energy'
ORDER BY Name
LIMIT 10
```

```
-- parent and child
SELECT Name, Account.Name FROM Contact
SELECT Name, (SELECT LastName FROM Contacts)
FROM Account
```

## 6 Big results

Results arrive in batches. If `done` is **false**, follow `nextRecordsUrl` until it is true.

## 7 Status codes

| Code | Meaning |
|---|---|
| 200 / 201 | OK / created |
| 204 | Success, no body (update, delete) |
| 400 | Bad request or validation error |
| 401 | Token expired: re-authenticate |
| 403 | No permission, or `REQUEST_LIMIT_EXCEEDED` |
| 404 | Record or URL not found |

## 8 Common mistakes

- Querying without `LIMIT` or filters
- One call per record in a loop (see page 7)
- Ignoring the `errors` array

## 9 Remember

**Verb + resource + JSON. Read the status code first.**

**Real-world use cases**
Web and mobile apps, scripts, middleware flows.

**Interview tip**
What does a 204 mean, and why does PATCH not return a body?

**Pro tip**
Use `/limits` to see how much of your daily API quota is left.

# Composite & Bulk API 2.0

APIs
Integration

*Fewer calls for many records*

## 1 Why fewer calls?

Every REST call costs ***latency*** and counts against your ***API quota***. Batch many operations into one call, or hand the job to Salesforce to process in the background.

## 2 The options

| Option | What it does |
|---|---|
| sObject Collections | Up to 200 records of one operation in one call |
| Composite | Up to 25 steps; later steps can use earlier results |
| Composite Batch | Up to 25 independent steps in one call |
| Bulk API 2.0 | Async jobs for large volumes (CSV) |

## 3 Composite example

```
POST /services/data/v67.0/composite
{ "allOrNone": true,
  "compositeRequest": [
   { "method": "POST",
     "referenceId": "refAcc",
     "url":
"/services/data/v67.0/sobjects/Account",
     "body": {"Name": "Acme"} },
   { "method": "POST",
     "referenceId": "refCon",
     "url":
"/services/data/v67.0/sobjects/Contact",
     "body": {"LastName": "Rao",
       "AccountId": "@{refAcc.id}"} } ] }
```

`@{refAcc.id}` reuses the new Account Id. `allOrNone` rolls everything back on any failure.

## 4 Bulk API 2.0 ingest flow

1. Create job → 2. Upload CSV → 3. Close job → 4. Poll status → 5. Get results

- `POST /jobs/ingest` to create
- `PUT .../batches` to upload CSV
- `PATCH` state to `UploadComplete`
- Check job, then read ***successful***, ***failed*** and ***unprocessed*** results

## 5 Common mistakes

- Looping single REST calls for thousands of records. Fix: Collections or Bulk.
- Skipping the ***failed results*** file. Fix: always download and review it.
- Mixing dependent steps into Composite Batch. Fix: Batch steps cannot share results.

## 6 Which one?

| Situation | Use |
|---|---|
| Under about 200 records | Collections |
| A few dependent steps | Composite |
| 10,000+ records | Bulk API 2.0 |

A guide, not a hard rule. Check current limits.

## 7 Remember

***Few records: Collections. Dependent steps: Composite. Millions: Bulk.***

**Real-world use cases**
Nightly data loads, migrations, creating a parent and children together.

**Interview tip**
Composite vs Composite Batch: can steps reference each other?

**Pro tip**
Use an External ID field so Bulk jobs can upsert instead of insert.

# Integration Patterns

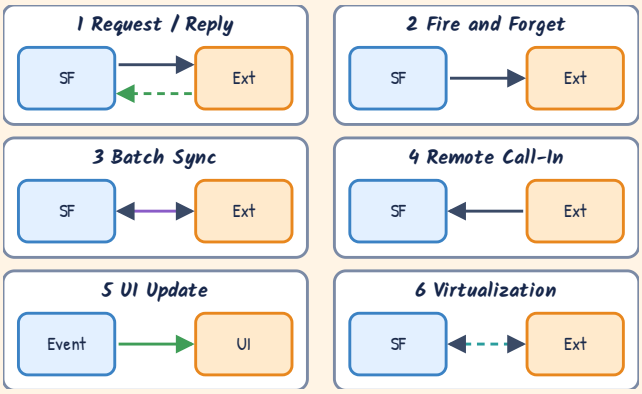
APIs
Integration

*The six standard ways to connect systems*

## 1 What is it?

Salesforce documents ***six standard integration patterns***. Choosing one early decides your timing, tools and failure handling.

## 2 The six at a glance




## 3 Pattern table

| Pattern | Timing | Example | Typical tools |
|---|---|---|---|
| Request and Reply | Sync, waits | Get a tax rate | Apex callout, External Services |
| Fire and Forget | Async | Send order to fulfilment | Events, outbound call |
| Batch Data Sync | Scheduled | Nightly ERP load | Bulk API, middleware |
| Remote Call-In | External starts | ERP updates Account | REST, Apex REST |
| UI Update on Data Change | Near real time | Live screen refresh | Platform Events, CDC |
| Data Virtualization | Live view | Show ERP invoices | Salesforce Connect |

## 4 Choose in 4 questions

1. ***Copy or just view?*** View only: virtualization.
2. ***Need the answer now?*** Yes: request and reply.
3. ***How much data?*** Millions: batch.
4. ***Who starts it?*** The other system: remote call-in.

## 5 Common mistakes

- Using request-and-reply for slow services. Fix: async or events.
- Copying data you only need to display. Fix: virtualize.
- One pattern for every case. Fix: mix patterns per need.

## 6 Remember

***Timing, volume, ownership: then pick the pattern.***

**Real-world use cases**
Design reviews, architecture diagrams, choosing between callouts and events.

**Interview tip**
Name the six patterns and one tool for each.

**Pro tip**
Draw the pattern on a whiteboard before opening Setup.

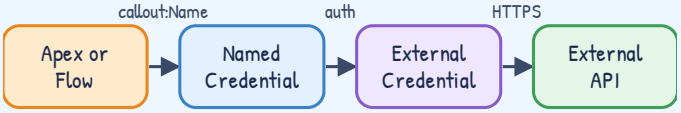
# Callouts: Named Credentials & External Services

APIs
Integration

*Calling out safely from Apex and Flow*

## 1 What is a callout?

A ***callout*** is an HTTP request that Apex or Flow sends ***out*** to an external service, such as a tax, address or payment API.




## 2 Named + External Credentials

***Named Credential:*** the endpoint URL.
***External Credential:*** how to authenticate (OAuth, token, and so on).
***Permission sets*** decide who can use them.

Result: no URLs or secrets in your code.

## 3 Apex callout

```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:Tax_Service/v1/rates?
zip=94105');
req.setMethod('GET');
HttpResponse res = new Http().send(req);
if (res.getStatusCode() == 200) {
    // parse res.getBody()
}
```

`callout:Name` points to the Named Credential.

## 4 Rules to know

- No callout in a trigger directly. Use ***Queueable*** with `Database.AllowsCallouts` or `@future(callout=true)`.
- No callout after ***uncommitted DML*** in the same transaction.
- Up to ***100*** callouts per transaction, ***120 s*** total timeout.

## 5 External Services

Import an ***OpenAPI*** schema and call the service from ***Flow*** with no Apex. Good for simple request-and-reply calls.

## 6 Test it

```
@isTest
global class TaxMock implements HttpCalloutMock
{
  global HTTPResponse respond(HTTPRequest r) {
    HttpResponse res = new HttpResponse();
    res.setStatusCode(200);
    res.setBody('{"rate":0.08}');
    return res;
  }
}
// in the test:
Test.setMock(HttpCalloutMock.class, new
TaxMock());
```

Tests cannot make real callouts, so always mock.

## 7 Common mistakes

- Hard-coding URLs and keys. Fix: Named + External Credentials.
- Callout inside a trigger. Fix: go async.
- Not handling non-200 responses. Fix: check status, log, retry rules.

## 8 Remember

***Name it, do not hard-code it.***

**Real-world use cases**
Tax, address validation, payment status, enrichment services.

**Interview tip**
Why can you not call out after DML in the same transaction?

**Pro tip**
Keep timeouts short and handle slow services with async calls.

# Event-Driven: Platform Events, CDC & Pub/Sub API

APIs
Integration

*Get notified instead of polling*

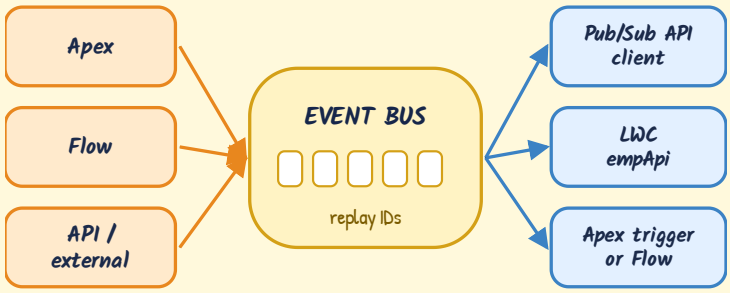
## 1 What is it?

Salesforce has an ***event bus***. Systems ***publish*** events to it and others ***subscribe***. Two main types: ***Platform Events*** (your own business events) and ***Change Data Capture (CDC)*** (automatic record-change events).

## 2 Why use events?

- Near-real-time updates without polling
- Loose coupling: many subscribers per event
- Fewer API calls, fewer limit problems

## 3 The event bus




**Channels:** `/data/ChangeEvents`, `/data/OpportunityChangeEvent`, `/event/Order_Placed__e`

## 4 Compare

| | Platform Event | CDC |
|---|---|---|
| Defined by | You (schema) | Salesforce |
| Fires when | You publish | Record created, updated, deleted, undeleted |
| Best for | Business events | Keeping systems in sync |

## 5 Worked example

1. Enable CDC for ***Opportunity***.
2. A subscriber connects with ***Pub/Sub API*** (gRPC over HTTP/2).
3. It subscribes to `/data/OpportunityChangeEvent`.
4. Each event carries a ***replay ID***, so it can resume after a disconnect.

## 6 Common mistakes

- Assuming events are kept forever. Fix: store the replay ID, reconnect quickly, check the retention window.
- Using Platform Events to copy records. Fix: use CDC.
- Duplicates not handled. Fix: idempotent subscribers.

## 7 Remember

***CDC = what changed. Platform Event = what happened. Pub/Sub API = how to listen.***

**Real-world use cases**
ERP sync, data lake feeds, notifications, order workflows.

**Interview tip**
What is a replay ID and why does it matter?

**Pro tip**
Pub/Sub API is for system integration, not for end-user screens.

# Apex REST: Inbound Custom Endpoints

APIs
Integration

*Expose your own API from Salesforce*

## 1 What is it?

**Apex REST** lets you publish your **own** API endpoint from Salesforce. Callers use it like any REST API, but your Apex code runs the logic.




## 2 Annotations

| Annotation | HTTP |
|---|---|
| `@HttpGet` | GET |
| `@HttpPost` | POST |
| `@HttpPatch` | PATCH |
| `@HttpPut` | PUT |
| `@HttpDelete` | DELETE |

URL: `/services/apexrest/{urlMapping}`

## 3 Example

```
@RestResource(urlMapping='/OrderSync/v1/*')
global with sharing class OrderSyncApi {
  global class Result {
    global Id accountId;
  }
  @HttpPost
  global static Result create(String name) {
    Account a = new Account(Name = name);
    insert a;
    Result r = new Result();
    r.accountId = a.Id;
    return r;
  }
}
```

Class and methods must be `global`; methods are `static`.

## 4 Custom or standard REST?

| Use | When |
|---|---|
| Standard REST | Plain CRUD on records |
| Apex REST | Custom logic, multi-object work in one call, special validation |

## 5 Secure it

- Use `with sharing`
- Enforce object and field access, for example `WITH USER_MODE` in SOQL
- Validate every input
- Call it as a least-privilege integration user

## 6 Common mistakes

- No input validation. Fix: reject bad data early.
- No version in the URL. Fix: `/v1/` in the mapping.
- Returning huge payloads. Fix: paginate.

## 7 Remember

***Standard REST first. Apex REST when you need custom logic.***

**Real-world use cases**
Webhooks from other apps, one-call order creation, custom validation.

**Interview tip**
When is Apex REST better than the standard REST API?

**Pro tip**
Version your endpoints so you can change them without breaking callers.

# Salesforce Connect & Middleware

APIs
Integration

*Virtual data and iPaaS*

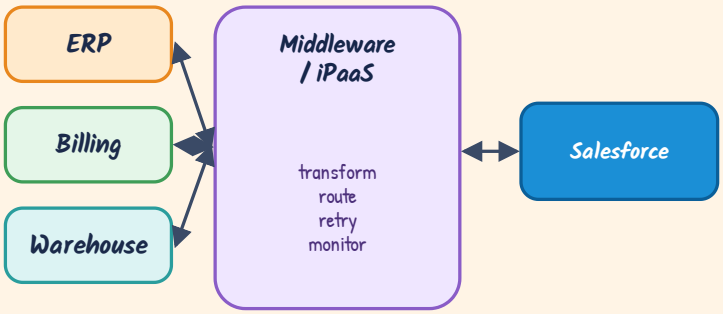
## 1 Salesforce Connect

Shows data that lives in another system ***without copying it***. The data appears as ***external objects*** (names end in `__x`) through ***OData***, a cross-org adapter or a custom Apex adapter.

## 2 Copy or virtualize?

| | Copy data | Virtualize |
|---|---|---|
| Reports and automation | Full support | Limited |
| Freshness | As of last sync | Live |
| Storage and sync | You manage | None |
| Best when | You need to act on it | You only need to see it |

## 3 Middleware (iPaaS)




Examples: ***MuleSoft*** (a Salesforce company), Boomi, Workato.

## 4 Point-to-point or middleware?

| | Point-to-point | Middleware |
|---|---|---|
| Systems | 2 | 3 or more |
| Transformation | Light | Heavy |
| Monitoring and retry | Build yourself | Built in |
| Cost | Low at first | Licence and skills |

## 5 Common mistakes

- Copying data you only display. Fix: Salesforce Connect.
- Putting business logic in many places. Fix: keep it in one layer.
- No monitoring on the middleware. Fix: alerts and dashboards.

## 6 Remember

***Copy what you must. Virtualize what you can.***

**Real-world use cases**
Showing ERP invoices on an Account, connecting many systems with one hub.

**Interview tip**
When do you choose middleware over direct integration?

**Pro tip**
Decide who owns each field before you design the sync.

# Limits & Governors

APIs
Integration

*Why integrations fail at scale*

## 1 Two kinds of limits

**1. API request limits** are counted **per org** over a rolling **24 hours**. They depend on your edition and licences.

**2. Apex governor limits** apply **per transaction** so no single process can hog shared resources.

## 2 Key numbers (Apex, per transaction)

| Limit | Value |
|---|---|
| Callouts (HTTP requests) | **100** |
| Total callout timeout | **120 seconds** |
| SOQL queries, synchronous | **100** |
| SOQL queries, asynchronous | **200** |
| Records retrieved by SOQL | **50,000** |

Numbers can change. Confirm in the current Apex Developer Guide.

## 3 Check your usage

```
GET /services/data/v67.0/limits
```

Returns remaining and maximum values, including `DailyApiRequests`.

API requests used (24 h, example)

Callouts in one transaction (example 40 of 100)

SOQL queries, sync (example 30 of 100)

## 4 Stay under the limits

- Use Collections, Composite and Bulk (page 7)
- Subscribe to events instead of polling (page 10)
- **Bulkify** Apex: no SOQL or callouts inside loops
- Move callouts to async jobs
- Monitor `/limits` and alert at 70–80%

## 5 Common mistakes

- SOQL or callout inside a loop. Fix: query once, loop over the results.
- Many integrations sharing one quota. Fix: track usage per integration.
- No back-off after REQUEST_LIMIT_EXCEEDED. Fix: pause and retry later.

## 6 Remember

***Limits protect everyone. Design for them, do not fight them.***

**Real-world use cases**
Capacity planning before a data migration or a new integration goes live.

**Interview tip**
Name three governor limits and how you avoid them.

**Pro tip**
Give each integration its own user so usage is easy to trace.

# Errors, Retries & Idempotency

APIs
Integration

*Safe re-sends with External IDs*

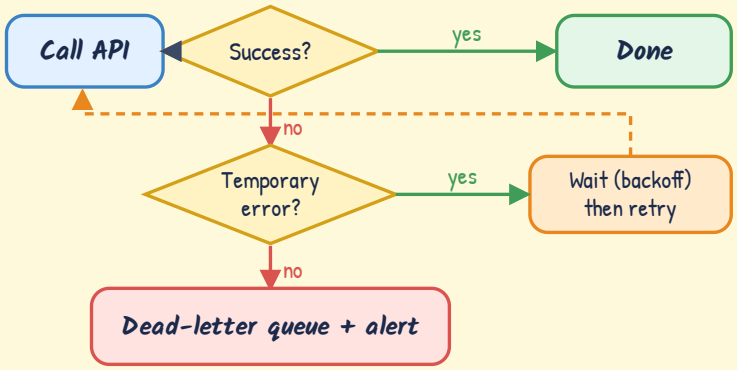
## 1 Why calls fail

- Network drops and timeouts
- Limits: `REQUEST_LIMIT_EXCEEDED`
- Record locks: `UNABLE_TO_LOCK_ROW`
- Validation errors in the data
- Expired token (401)

## 2 What to do

| Error | Action |
|---|---|
| 401 | Re-authenticate, retry once |
| 403 limit | Back off, reduce calls |
| 5xx, timeout | Retry with backoff |
| 400 validation | Do not retry: log, park, fix data |
| Row lock | Retry later, lower parallelism |

## 3 Retry flow




## 4 Idempotency with External ID

```
PATCH /services/data/v67.0/sobjects/
      Order__c/Ext_Order_Id__c/ORD-1001
{ "Status__c": "Paid" }
```

Run it twice, you still get **one** record. First call returns **201** (created), later calls **204** (updated).

## 5 Backoff formula

```
wait = base * 2^attempt + random jitter
# e.g. 1s, 2s, 4s, 8s ... then give up
```

Cap the number of attempts.

## 6 Common mistakes

- Retrying forever. Fix: cap attempts, then dead-letter.
- Retrying errors that can never succeed. Fix: only retry temporary ones.
- Insert on retry, creating duplicates. Fix: upsert by External ID.
- No correlation Id in logs. Fix: log one per request.

## 7 Remember

**Retry the temporary. Park the permanent. Upsert by External ID.**

**Real-world use cases**
Reliable order sync, payment updates, any integration that must not lose or duplicate data.

**Interview tip**
What is idempotency and how do you get it in Salesforce?

**Pro tip**
Store the failed payload with the error so support can replay it.

# Security Best Practices

APIs Integration

*Least privilege, protected secrets, full audit trail*

## 1 Least privilege

- A ***dedicated integration user*** per integration
- Minimal ***permission set***, not System Administrator
- Grant only the objects and fields it needs

## 2 Authentication

- OAuth with an ***External Client App***
- Request minimal scopes
- Rotate secrets and certificates
- Retire SOAP `login()` and password flows

## 3 Defence in layers

Network: TLS, IP allowlist

Identity: OAuth, ECA, integration user

Permissions: permission set, field access

Code: with sharing, validate input

Audit: logs and monitoring

## 4 Secrets

- Use ***Named / External Credentials***
- Environment variables or a secrets manager outside Salesforce
- Never in source code, logs or error messages

## 5 Data

- TLS for every call
- Send only the fields that are needed
- Consider ***Shield Platform Encryption*** for sensitive fields

## 6 Apex and endpoints

- `with sharing` and `WITH USER_MODE`
- Validate every incoming value
- Never trust the payload

## 7 Watch it

- Review ***Login History*** and ***Setup Audit Trail***
- Alert on failed logins and limit spikes
- Use Event Monitoring if licensed

## 8 Do and do not

| Do | Do not |
|---|---|
| Use OAuth and ECAs | Share one admin login |
| Store secrets in credentials | Hard-code keys |
| Log request Ids | Log passwords or tokens |

## 9 Remember

***Least privilege, no secrets in code, log everything.***

**Real-world use cases**
Security reviews, audits, onboarding a new integration partner.

**Interview tip**
How do you secure an integration user end to end?

**Pro tip**
Review integration users and permissions every quarter.

# Testing & Tooling

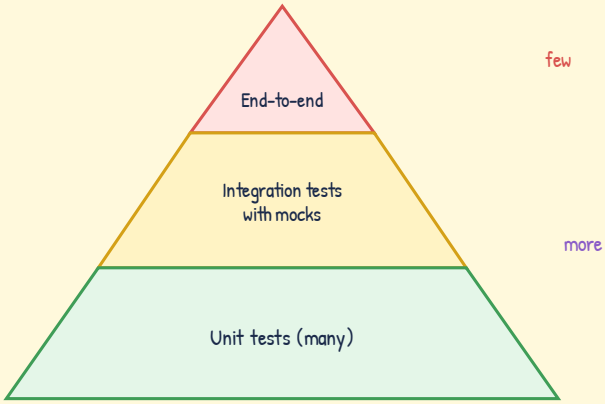
APIs Integration

*Mocks, sandboxes, the CLI and Postman*

## 1 Your toolbox

- **Developer Edition** org or sandbox
- **Postman** or curl for API calls
- **Salesforce CLI** (`sf`)
- VS Code with Salesforce extensions
- Debug logs and the Limits resource

## 2 Test pyramid




## 3 Mock a callout

```
@isTest
global class TaxMock implements HttpCalloutMock {
  global HTTPResponse respond(HTTPRequest r) {
    HttpResponse res = new HttpResponse();
    res.setStatusCode(200);
    res.setBody('{"rate":0.08}');
    return res;
  }
}
// Test.setMock(HttpCalloutMock.class, new
TaxMock());
```

## 4 CLI quick start

```
sf org login web
sf data query -q "SELECT Id FROM Account LIMIT 1"
sf apex run test
```

## 5 Postman steps

1. Request a token from `/services/oauth2/token`
2. Set **Authorization: Bearer**
3. Call `/limits` first, then your real endpoint
4. Save as a collection to share

## 6 Test checklist

- Happy path
- Expired token
- Rate limit reached
- Duplicate message
- Bad data
- Large volume

## 7 Common mistakes

- Testing only the happy path
- Using production data in a sandbox without masking

## 8 Remember

**Mock the outside world. Test the failure paths.**

**Real-world use cases**
Releasing integrations safely and catching failures before users do.

**Interview tip**
How do you test Apex that makes callouts?

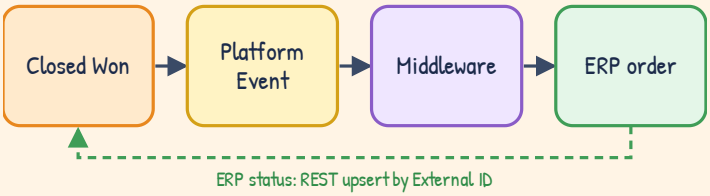
**Pro tip**
Keep a Postman collection per integration and store it with the code.

# Real-World Projects

APIs
Integration

*Three blueprints you can copy*

## A Salesforce and ERP (orders)

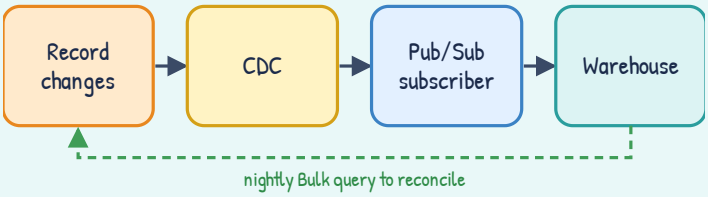



- ***Patterns:*** fire and forget, remote call-in
- ***Tools:*** Platform Events, middleware, REST upsert
- ***Risk:*** duplicate orders. ***Fix:*** External ID

***Steps***

1. Opportunity reaches Closed Won
2. Flow or trigger publishes an Order event
3. Middleware maps fields and creates the ERP order
4. ERP status returns by REST upsert (External ID)

## B Salesforce to data warehouse

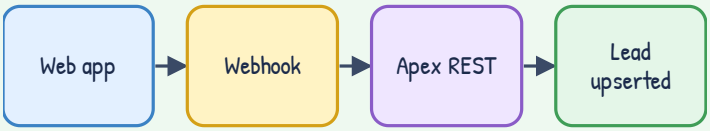



- ***Patterns:*** batch sync, UI update on data change
- ***Tools:*** CDC, Pub/Sub API, Bulk API 2.0
- ***Risk:*** missed events. ***Fix:*** replay ID and a nightly check

***Steps***

1. Enable CDC on the objects you need
2. Subscriber connects to Pub/Sub API with its last replay ID
3. Events are written to the warehouse
4. A nightly Bulk query checks for gaps

## C Web app to Salesforce (leads)




- ***Patterns:*** remote call-in
- ***Tools:*** Apex REST, External ID upsert
- ***Risk:*** bad input. ***Fix:*** validation and a dead-letter log

***Steps***

1. Web form posts to a webhook
2. Webhook calls the Apex REST endpoint
3. Apex validates and upserts the Lead by External ID
4. Errors go to a log for replay

## Project checklist

1. Pick the pattern and write it down
2. Choose the API and auth (ECA)
3. Estimate volume against limits
4. Add retries, idempotency, logging
5. Mock and test failure paths
6. Review security and permissions
7. Monitor after go-live

## Remember

***Pattern, API, auth, limits, errors, security: in that order.***

**Real-world use cases**
Order to cash, analytics feeds, lead capture from websites.

**Interview tip**
Walk me through an integration you designed, end to end.

**Pro tip**
Draw the diagram first. The code gets easier.

# Cheat Sheet & Interview Guide

APIs Integration

*Quick revision of the whole series*

## 1 API choice

Small: REST
Big: Bulk 2.0
Notify: Pub/Sub

## 2 Auth

OAuth + ECA
Client Credentials for servers
No new Connected Apps

## 3 REST verbs

POST create
GET read
PATCH update
DELETE delete

## 4 SOQL

`SELECT fields`
`FROM Object WHERE ...`
follow `nextRecordsUrl`

## 5 Fewer calls

Collections: 200 records
Composite: 25 steps
Bulk 2.0: large jobs

## 6 Patterns

Request/Reply, Fire/Forget, Batch, Call-In, UI Update, Virtualization

## 7 Callouts

`callout:Name`
100 per transaction
120 s total timeout

## 8 Events

Platform Event = happened
CDC = changed
Pub/Sub API = listen

## 9 Apex REST

`@RestResource`
`@HttpPost` ...
with sharing, version URL

## 10 Limits

Per-org API quota (24 h)
Per-transaction governors
Check `/limits`

## 11 Errors

Retry temporary
Park permanent
Upsert by External ID

## 12 Security

Least privilege
Secrets in credentials
Audit everything

## Top 10 interview questions

1. REST vs SOAP vs Bulk: when do you use each?
2. How does the Client Credentials flow work? What is the run-as user?
3. Platform Events vs Change Data Capture?
4. What is idempotency and how do you get it?
5. Why can you not call out after DML?
6. How do you avoid hitting API limits?
7. Why use a Named Credential?
8. When would you choose middleware?
9. How do you secure an integration user?
10. How do you handle failures and retries?

## Final formula

**Pattern → API → Auth → Limits → Errors → Security**

**Real-world use cases**
Last-minute revision before an interview or design review.

**Interview tip**
Answer with the pattern first, then the API, then the failure handling.

**Pro tip**
Re-check pages 4, 5 and 15 against the latest release notes.